

Memoria

Programación multimedia y dispositivos móviles

José Miguel Gutiérrez López

Índice

Parte A · Análisis del desarrollo móvil **3**
Ventajas e inconvenientes 3
Elección tecnológica 5
Ciclo de vida de una Activity 6
Parte B · Especificación de tu app **7**
Idea y público objetivo 7
Requisitos funcionales 8
Requisitos no funcionales 9
Casos de uso 9
Parte C · Entorno de desarrollo **11**
Creación del proyecto 11
Configuración de dispositivos 11
Ejecutar la app 12
Estructura del proyecto 14
Parte D · Recursos de la aplicación **14**
App en varios idiomas 14

Parte A · Análisis del desarrollo móvil

Ventajas e inconvenientes

Ventajas:

- **Portabilidad:** Permite tener muy buena portabilidad, ya que el móvil siempre va con la persona y puede usar la aplicación en cualquier momento, mientras que un portátil no suele estar siempre con la persona y es menos accesible.

- **Interfaz táctil:** El desarrollo en dispositivos móviles permite una interfaz táctil en la que el usuario pulsando y deslizando la pantalla puede ir moviéndose en la aplicación, mientras que en un portátil lo normal es usar ratón y teclado.

- **Notificaciones:** En caso de que haya novedades en la aplicación o cualquier cambio te notifica al instante y estás informado de cualquier cosa. Como esta app es de recetas que se hacen rápido, te notificarán cuando hayan nuevas recetas disponibles

- **Distribución:** Las aplicaciones están en apps como Google Play que son accesibles por todo el mundo y entonces llegas a mucha gente sin tener que montar nada.

Inconvenientes:

- **Pruebas en muchos dispositivos:** Hay muchos distintos tipos de móviles, con tamaños distintos o versiones de Android distintas, por lo que para que nuestra aplicación funcione en todos, antes hay que probar todo tipo de móviles y que nuestra aplicación funcione, en cambio una web funciona en cualquier navegador y no habría que hacer tantas pruebas.

- **Desarrollar para varios sistemas:** Si desarrollamos una aplicación de Android, no funcionará en iPhone, entonces para que nuestra aplicación sea accesible para toda la gente posible habría que hacer otra aplicación desarrollada en otro sistema, mientras que en un portátil como comente antes una web funciona en cualquier navegador, por lo que se tendría que desarrollar menos.

- **Seguridad y permisos:** En algunas aplicaciones se piden permisos al usuario para usar la cámara, ubicación... del propio usuario, aunque en portátil también existen los permisos es menos frecuente.

Limitaciones:

- **Batería:** Los dispositivos móviles tienen una batería limitada, entonces una aplicación que mantiene la pantalla encendida o una aplicación que pueda trabajar en segundo plano, hace que se gaste la batería más rápido. En mi aplicación de recetas el usuario puede estar más de 25 minutos viendo una receta con la pantalla encendida, por lo que eso gastaría bastante batería.

- **Pantallas más pequeñas:** Los dispositivos móviles tienen la pantalla más pequeña que un ordenador, por lo que en mi aplicación de recetas, tiene que leerse todo muy bien hasta en un móvil chico, así que habría que priorizar el contenido importante, ya que no cabe mucho texto.

- **Conexión:** Si la aplicación necesita internet, el usuario puede tener una conexión que va y viene y puede que no pueda utilizar la aplicación, o no todas las funciones de esa aplicación. En mi aplicación si para consultar las recetas tiene que consultar internet, no podría acceder a ellas si no tiene.

- **Memoria y CPU:** Los móviles tienen menos memoria y potencia que un ordenador, y el sistema puede cerrar una aplicación si necesita recursos o si no los tiene para que la aplicación funcione correctamente. En mi aplicación de recetas, si el usuario sale un momento para contestar un mensaje mientras cocina, la aplicación podría cerrarse y perdería el paso de la receta en el que iba.

Elección tecnológica

	Nativo	Híbrido	Multiplataforma
Lenguajes	Java o Kotlin en Android, y Swift en iOS.	HTML, CSS y JavaScript.	Dart, JavaScript/TypeScript, C#, Kotlin.
Rendimiento	Es el que tiene mejor rendimiento, ya que el código se comunica directamente con el sistema.	Es el que tiene el rendimiento más bajo, porque es una web que se ejecuta dentro de la app.	Alto, casi nativo, aunque hay una capa que adapta el código a cada sistema.
Acceso al hardware	Es total y directo.	Es limitado, con plugins.	Tiene un acceso amplio, pero con algún límite.
Coste	Más alto si se quiere varios sistemas, para más aplicaciones.	Más bajo.	Intermedio.
Framework	Android Studio.	Ionic.	Flutter.

Mi aplicación se desarrollará en nativo Android, porque ofrece un muy buen rendimiento y también tiene acceso directo al hardware del móvil, es decir, puede usar sus funciones sin usar ningún plugin, algo que es muy útil en mi aplicación, por ejemplo para las notificaciones de nuevas recetas.

Ciclo de vida de una Activity

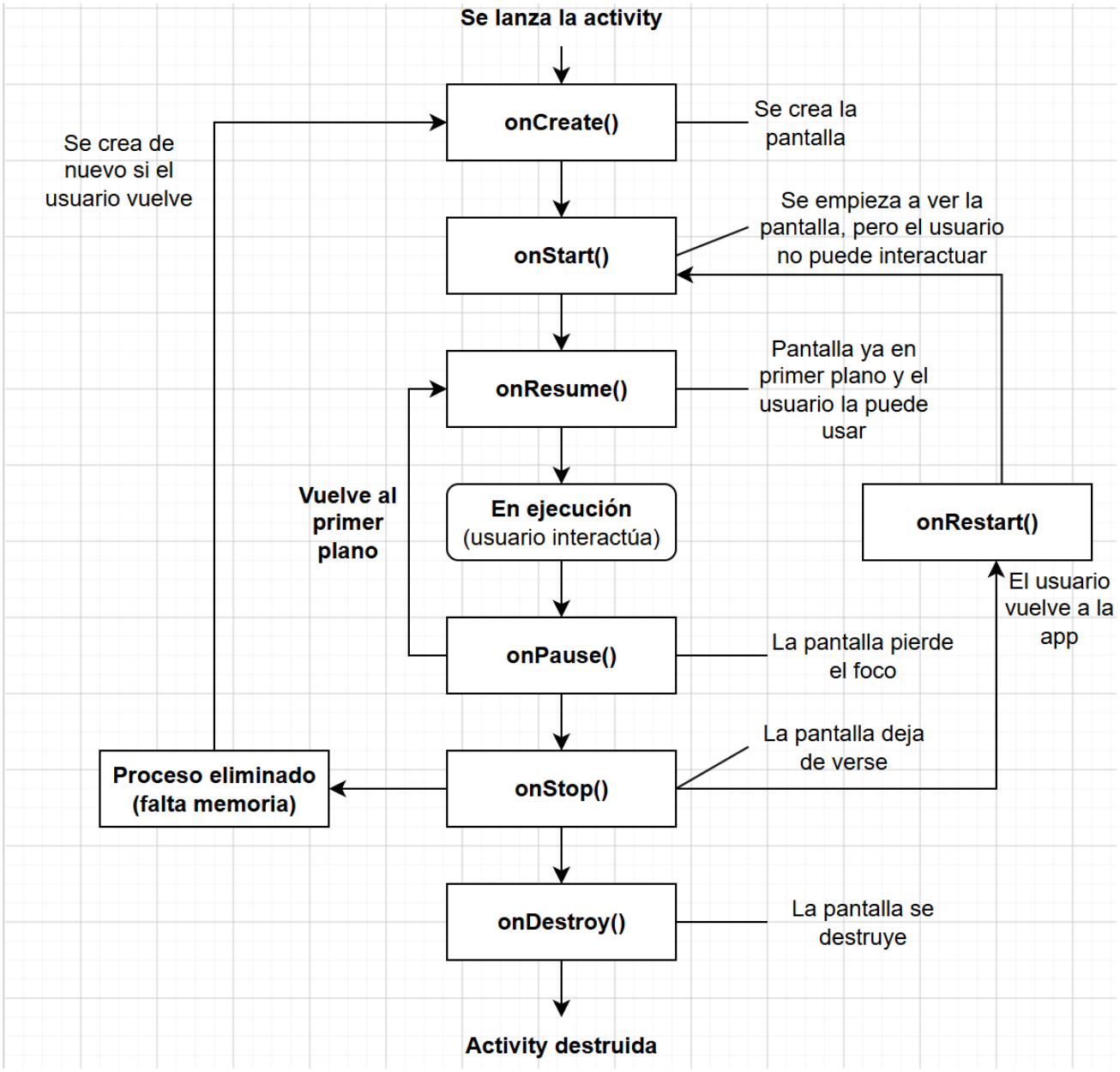


Figura 1. Ciclo de vida de una Activity

Si el usuario recibe una llamada:

Si el usuario recibe una llamada, lo primero que ocurre es que la pantalla pierde el foco, que es como si pasara a segundo plano, por lo que el usuario ya no puede interactuar con ella. Como la llamada llega a ocupar la pantalla entera, la activity deja de ser visible. Una vez se finaliza la llamada, el usuario vuelve a la aplicación.

Este caso pasa por los métodos: **onPause(), onStop(), onRestart(), onStart() y onResume(),** siguiendo este mismo orden.

Si el usuario gira el móvil:

Al girar el móvil se produce un cambio de configuración en el móvil, por lo que la activity se destruye, entonces Android crea una nueva que se adapte a la orientación nueva en la que esté el móvil.

Este caso pasa por los métodos: **onPause(), onStop(), onDestroy(), onCreate(), onStart() y onResume(),** siguiendo este mismo orden.

Si el usuario pulsa el botón Atrás:

Cuando el usuario pulsa el botón Atrás, la activity actual se cierra y deja de mostrarse en pantalla. Android la destruye porque ya no se va a utilizar. Si había una activity anterior, esta vuelve a mostrarse, si no, la aplicación se cierra. En mi aplicación, por ejemplo, en caso de que esté dentro de una receta, volvería a la lista de recetas

Este caso pasa por los métodos: **onPause(), onStop() y onDestroy(),** siguiendo este mismo orden.

Parte B · Especificación de tu app

Idea y público objetivo

RecetasRápidas es una aplicación Android, que como su propio nombre indica, es de recetas que son rápidas de hacer y también baratas, son creadas para un público más joven que no tienen tanta experiencia en la cocina y están comenzando a vivir solos, o para usuarios que tengan poco tiempo para cocinar. Las recetas de esta aplicación como mucho tardas en hacerlas 30 minutos, en las que no habría que usar muchos ingredientes y sin muchos pasos que hacer para terminar la receta. La aplicación permite a los usuarios buscar recetas, seguirlas con los paso a paso en la cocina y también guardar las recetas en favoritos, para que puedan verlas sin conexión, y te avisa si hay recetas nuevas.

Objetivos medibles:

- Que los usuarios accedan a las recetas en 3 toques o menos desde la pantalla principal de la aplicación, para que no pierdan mucho tiempo en ello.

- Que todas nuestras recetas que estén disponibles para los usuarios, se tarden menos de 30 minutos en hacerlas.

- Ofrecer a nuestros usuarios una interfaz limpia y clara, ya que así les satisface el usar nuestra aplicación porque les resulta fácil de usar.

Requisitos funcionales

- RF-01: El usuario puede ver la lista de recetas que están disponibles.
Prioridad: Alta

- RF-02: Que el usuario pueda entrar en la receta para ver los ingredientes y el paso a paso.
Prioridad: Alta

- RF-03: Que el usuario pueda guardar las recetas en favoritos.
Prioridad: Media

- RF-04: La aplicación recuerda el paso de la receta por el que iba el usuario.
Prioridad: Media

- RF-05: La aplicación envía notificaciones cuando haya una receta nueva.
Prioridad: Baja

Requisitos no funcionales

- RNF-01: Que todos los botones sean por lo menos de 48 dp y el texto de por lo menos 16 sp, para que usen la aplicación fácilmente y se vea todo claro.
Tipo: Usabilidad

- RNF-02: Que la aplicación funcione desde Android 7.0 (API 24) o superior, ya que es la versión que cubre la gran mayoría de móviles en uso.
Tipo: Compatibilidad

- RNF-03: Que cuando los usuarios abran la aplicación, la pantalla principal cargue en menos de 2 segundos y que cuando abran las recetas se abran también en menos de 2 segundos.
Tipo: Rendimiento

- RNF-04: Que la aplicación esté disponible en español y en inglés, y que el idioma se cambie según el idioma del dispositivo.
Tipo: Localización

Casos de uso

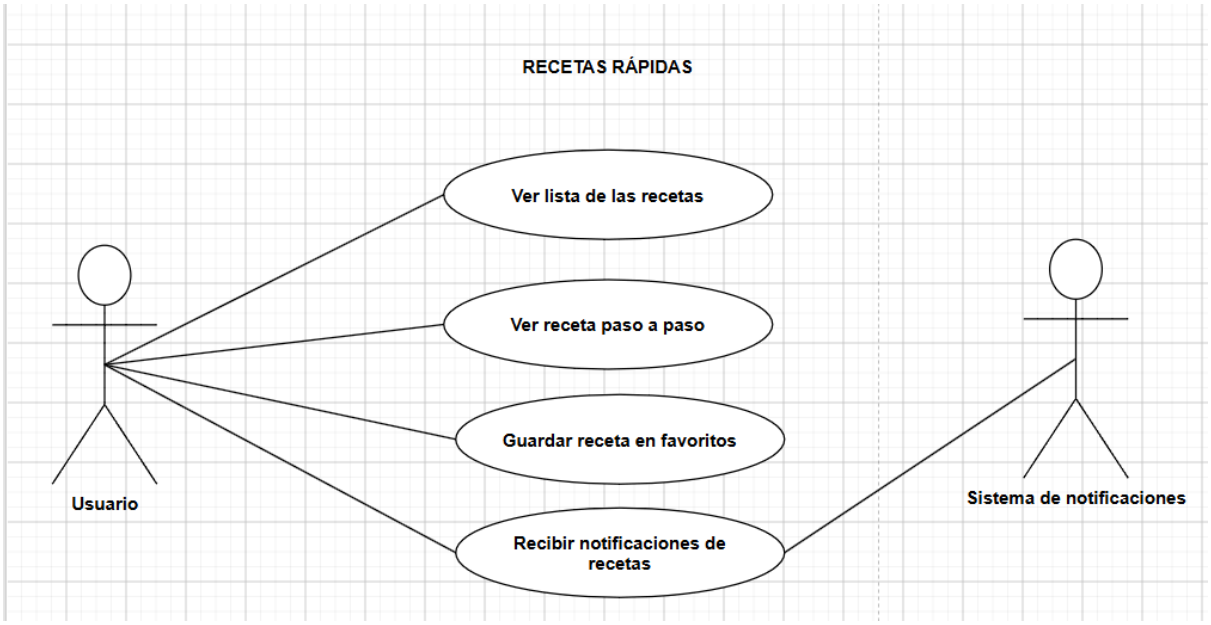


Figura 2. Diagrama casos de uso

Ficha detallada 1: Guardar en favoritos

- Actor: Usuario.
- Precondición: El usuario tiene abierta la pantalla de una receta.
- Flujo principal: 1. El usuario abre una receta.
2. El usuario pulsa el botón "Favoritos".
3. La aplicación guarda la receta en la lista de favoritos.
4. La aplicación muestra un mensaje de confirmación.
- Flujo alternativo: Si la receta ya estaba en favoritos, al pulsar el botón la app la quita de favoritos y el botón vuelve a su aspecto normal.
- Postcondición: La receta está en la lista de favoritas y puede verse sin conexión.

Ficha detallada 2: Ver receta paso a paso

- Actor: Usuario
- Precondición: La aplicación está abierta y hay recetas disponibles.
- Flujo principal: 1. El usuario accede a la lista de recetas.
2. El usuario selecciona una receta.
3. La aplicación muestra los ingredientes.
4. La aplicación muestra los pasos de elaboración.
5. El usuario consulta la información.
6. La aplicación guarda el paso en el que va el usuario.
- Flujo alternativo: Si el usuario sale de la app (por ejemplo, por una llamada) y vuelve más tarde, la app abre la receta en el último paso en el que iba.
- Postcondición: El usuario ha completado la receta o queda guardado el paso en el que iba.

Parte C · Entorno de desarrollo

Creación del proyecto

Lo primero de todo es crear es crear el proyecto en Android Studio con mi nombre y con un paquete propio.

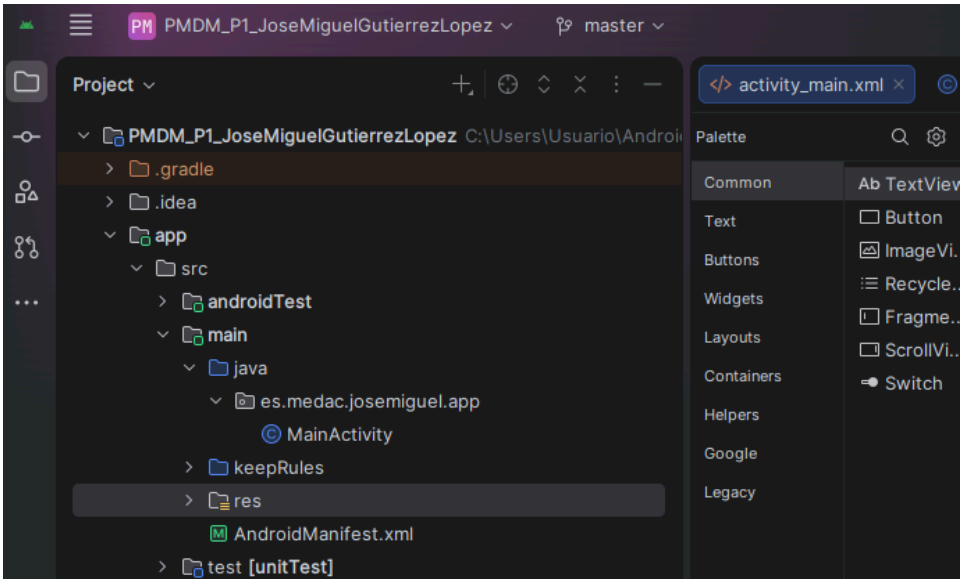


Figura 3. Creación del proyecto

Configuración de dispositivos

A la derecha del todo, vemos varios iconos, pues accedemos al que tiene el nombre de “Device Manager”, para añadir los dispositivos virtuales. Luego para ello, hacemos clic en el icono del “+”, y seleccionamos la opción de “Create Virtual Device”, y luego seleccionamos los dispositivos que queremos usar.

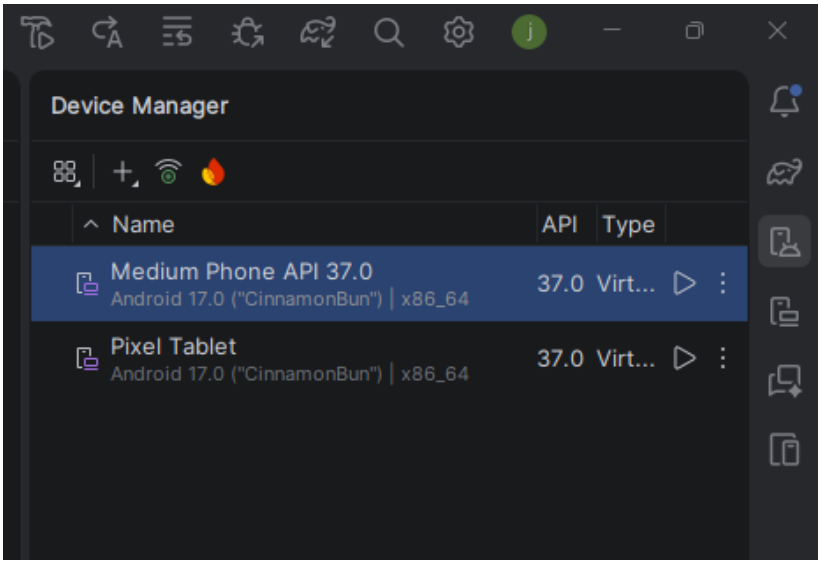


Figura 4. Dispositivos agregados

Dispositivo	Tipo	Resolución	Densidad	API
Medium Phone API 37.0	Móvil	1080 × 2400 px	420 dpi	37.0
Pixel Tablet	Tablet	2560 x 1600 px	320 dpi	37.0

Ejecutar la app

Vamos a ejecutar la aplicación, en los dos dispositivos, tanto en el móvil, como en la tablet.
Primero lo ejecuto en el móvil, y como podemos ver funciona perfectamente.

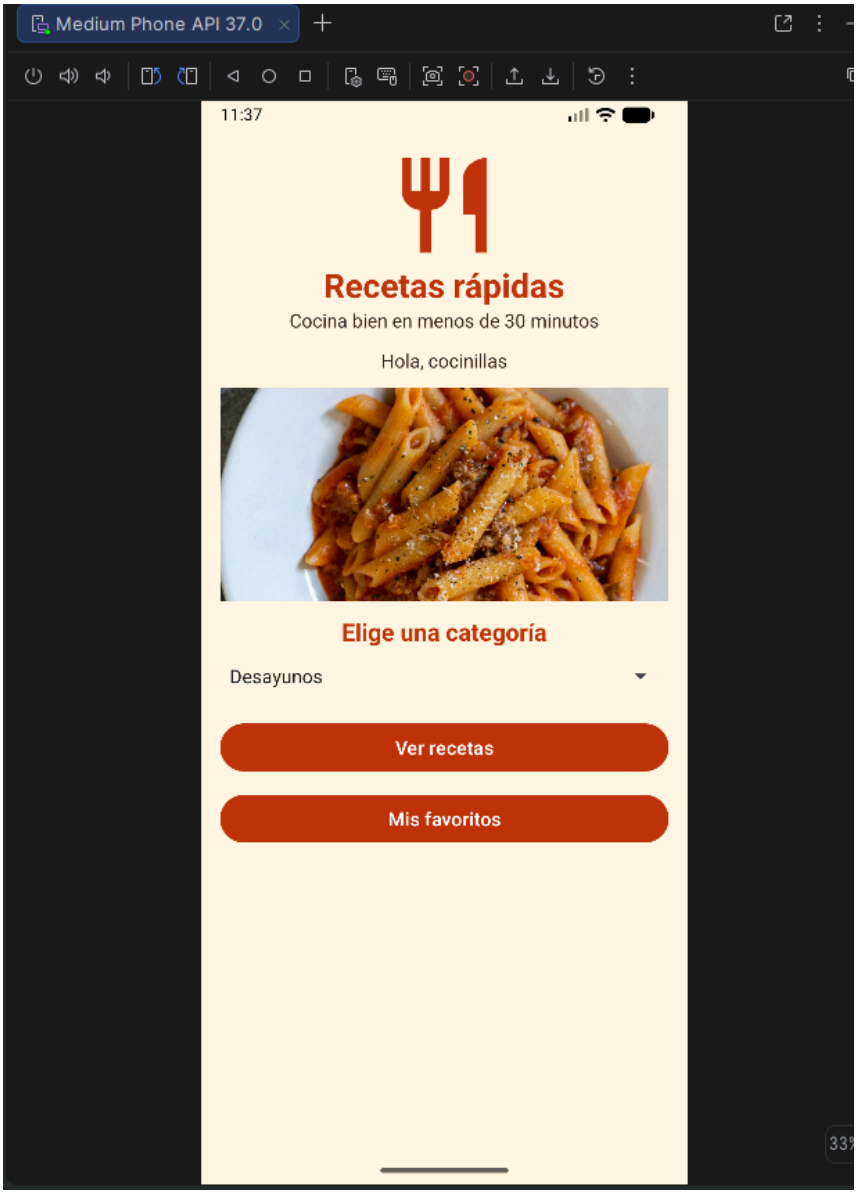


Figura 5. Ejecución de la app en móvil

Ahora lo voy a ejecutar en la tablet, y como se puede ver también funciona correctamente y se ve la pantalla de la aplicación.

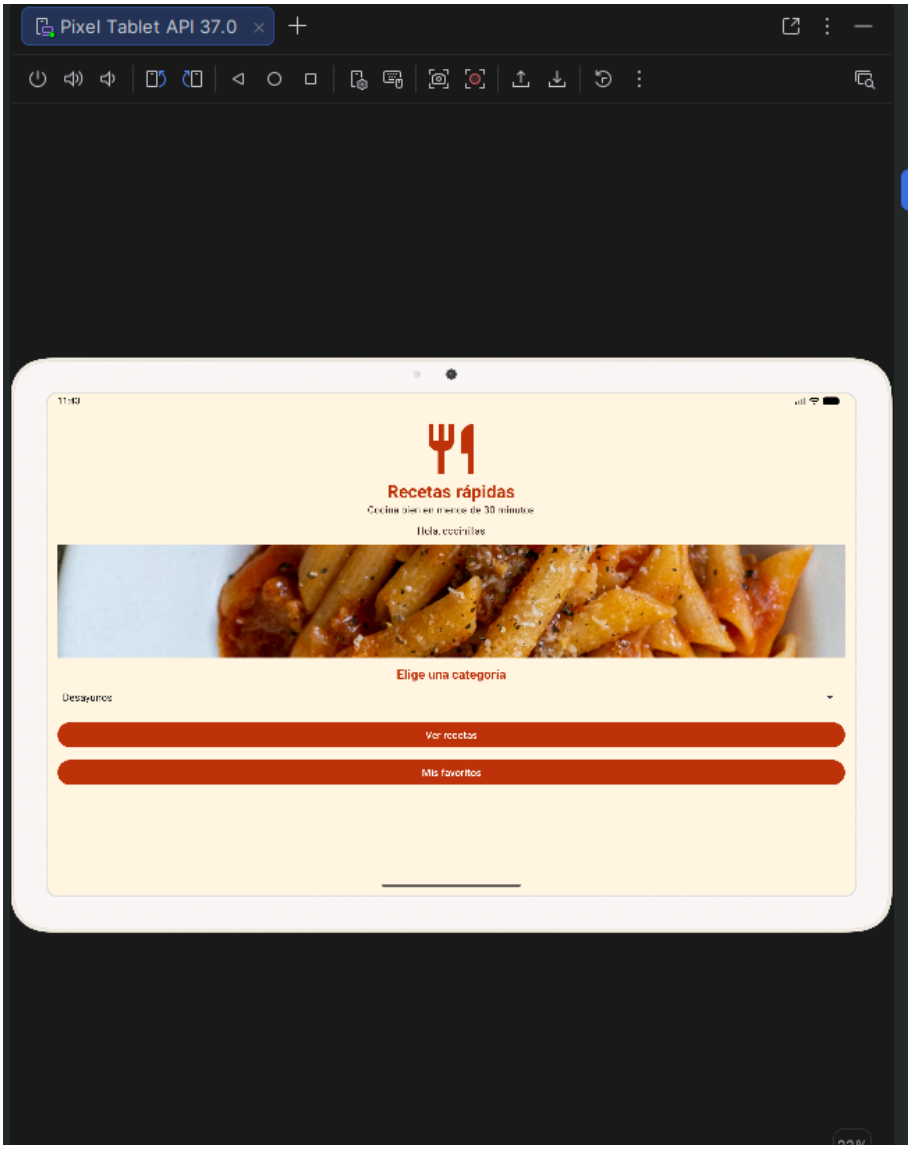


Figura 6. Ejecución de la aplicación en tablet

Además podemos ver el icono de mi aplicación, entre las aplicaciones instaladas en el móvil.

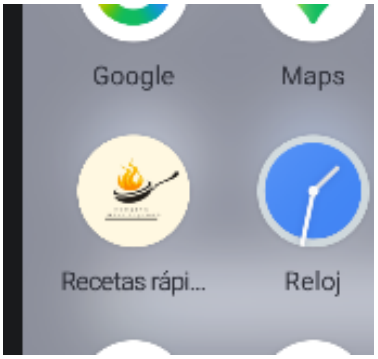


Figura 7. Icono en el lanzador

Estructura del proyecto

- AndroidManifest.xml: Es como la ficha de la aplicación, ya que este archivo, contiene la configuración general de la aplicación como el nombre, también contiene el icono, el tema, los permisos, las pantallas. Además, indica la activity que se abre cuando pulsamos el play.

- java/: Contiene el código Java de la aplicación, incluyendo las activities y la lógica del programa.

- res/: Contiene todo lo que no es código, como diseños, textos, colores, imágenes e iconos.

- Gradle: Los ficheros Gradle, gestionan las dependencias, librerías y configuración de compilación del proyecto.

- Relación Activity y Layout: La Activity decide qué hace la pantalla y el layout, que es el archivo XML, define cómo se ve.

Parte D · Recursos de la aplicación

App en varios idiomas

Aquí se puede ver la aplicación en español.

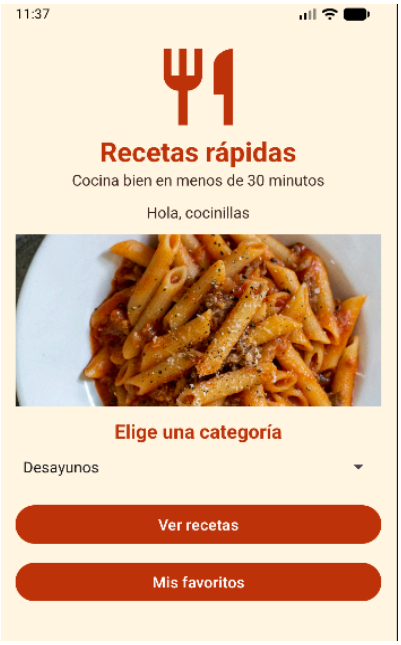


Figura 8. Aplicación en español

Y aquí también la podemos ver pero en inglés.

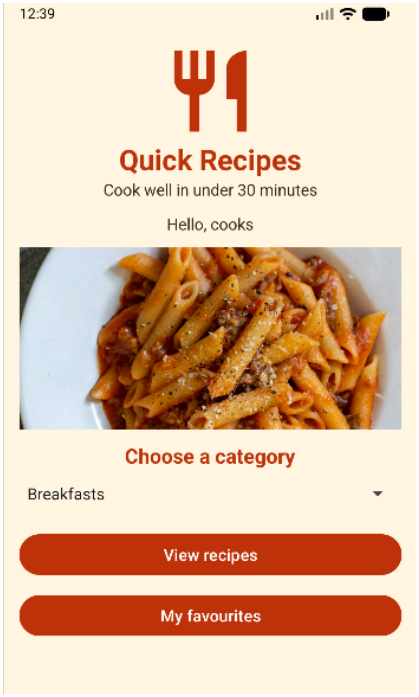


Figura 9. Aplicación en inglés